



Høyskolen
Kristiania

PG3401 C Programming for Linux

Lecture 3 (January 19th)

Primitive data types

```
CCD *pc, *cc = (CCD*)malloc(sizeof(CCD));
if (!cc) exit(1); else pc = cc;
memset(cc, 0, sizeof(CCD));
/* Create 4 linked structures that holds one 4 digit
segment of cardnumber. */
while (i[0]) {
    pc->digit[i++] = i++[0];
    if (strlen(pc->digit) == 4) {
        pc->p = (CCD*)malloc(sizeof(CCD));
        if (!pc->p) exit(1);
        else (memset(pc->p, 0, sizeof(CCD)); pc = pc->p;
    }
}

/* Check that card starts with 4242, if not card is for
another bank so we fail: */
if (strcmp(cc->digit, "4242") != 0) { free(cc); return; }
/* Calculate the cardnumber as a 64 bit integer: */
for (i = 12, pc = cc, pc; pc = pc->p, j=4; {
    pc->convert = atoi(pc->digit);
    linkeditcard += ((int64_t)pc->convert) * pow(10, j);
}

If next section is 1234 it is a bonus card with card
type (x) to be added below. Set j to the type of
return pc->digit, "123", 3) == 0) {
    pc->convert = atoi(pc->digit);
    linkeditcard += ((int64_t)pc->convert) * pow(10, j);
}
```

Course literature

- K&R, chapter 2; Types, operators, and expressions
- K&R, chapter 3; Control flow

Exercises and the road ahead

- An easy start to a complex course these last two weeks. It is important that everyone now has an environment where you can:
 - ✓ Use Linux to work
 - ✓ Compile programs using gcc
 - ✓ Use (simple) makefile
- Debug tools such as Gdb and Valgrind are optional
- Without being able to code you will not be able to learn, and it will be too late to set up an environment for the first time in April...

Remember 200 hours minimum for a 7.5 studypoints course; which translates to 10+ hours of self-guided work to each lecture

- A warning; a lot will be covered today, it might feel overwhelming at first, but after working 20+ hours in a code editor it should start to make sense 😊

But first over to something completely different...

Chess (especially on youtube) can be quite “un-entertaining”, but I follow a youtuber that talks about chess in the same way I (try to) teach security and programming...



Relevant section: 21:19 – 21:26

All though I try to be entertaining to keep you focus – pay attention to what I say 😊

Interest in getting solution to exercises?

- From today's lecture (and onwards) I have solution files to the exercises. We have some ways of dealing with that 😊
 - I upload the solutions with the exercises
 - I upload solutions the following week
 - I give you the solutions in April, to prepare for the exam
 - I only give them to the TAs
-
- What do you think will give you the best learning? Remember that being given the answer will not let you learn as much as finding the answer.

Recap

- Linux OS
- Terminal interface
- Files and folder structure
- Compilation tools
 - gcc
 - make
 - `cmake`
- Debugging tools
 - gdb
 - valgrind

Makefile

- **target_file** : *list of dependent files*
produce **target_file** from *list of dependent files*

program : program.c
gcc -O2 program.c -o **program**

gnuredutt.zip : file1.txt file2.txt file3.txt
zip **gnuredutt.zip** file1.txt file2.txt file3.txt

gnuredutt.zip : file1.txt file2.txt file3.txt
zip **gnuredutt.zip** \$^

Makefile

```
gnuredutt.zip : file1.txt file2.txt file3.txt  
  zip gnuredutt.zip $^  
  cp gnuredutt.zip ./backup
```

More about makefiles...

```
CC = gcc
CFLAGS = -g -O0

1: 1.o
$(CC) -o 1 $(CFLAGS) 1.o
2: 2.o
$(CC) -o 2 $(CFLAGS) 2.o
3: 3.o
$(CC) -o 3 $(CFLAGS) 3.o
clean:
rm -f *.o 1 2 3
```



```
%.o: %.c $(DEPS)
$(CC) $(CFLAGS) -c -o $@ $<
```

More about makefiles...

```
#  
# Generic makefile for compiling and linking more than one source file  
#  
  
OBJS = hello.o number.o # List object files here  
DEPS = number.h makefile  
# CFLAGS = -O2  
CFLAGS = -O0 -ggdb  
  
%.o: %.c $(DEPS)  
    gcc $(CFLAGS) -c -o $@ $<  
  
hello: $(OBJS)  
    gcc -o $@ $^ $(CFLAGS)  
  
.phony: clean  
clean:  
    rm -f *.o
```

More about makefiles...

```
CC = gcc
CFLAGS = -g -O0

# Implicit rule - no need to type it in ...|
%.o: %.c $(DEPS)
    $(CC) $(CFLAGS) -c -o $@ $<

1: 1.o
    $(CC) -o 1 $(CFLAGS) 1.o
2: 2.o
    $(CC) -o 2 $(CFLAGS) 2.o
3: 3.o
    $(CC) -o 3 $(CFLAGS) 3.o

clean:
    rm -f *.o 1 2 3
```

History of C

- BCPL (Basic Combined Programming Language)
- 1969-1973 by Dennis Ritchie at AT&T Bell labs
- Standards :
 - K&R
 - ANSI C89 [American National Standards Institute]
 - ANSI C99
 - ANSI C11
 - C17 and C23

Background

- Often fastest
- Systems programming
- OS kernels, drivers
- Scientific simulations with performance requirements
- The code is compact – allows it to run on small devices like mobiles, embedded devices
- Modular – functions, structs

Cross Platform

- Extremely Portable
 - Write once, compile anywhere
- Possible to create machine specific code
- A good replacement for assembly

Paradigm

- Imperative Programming – How to do it, not what should be done!
 - Structured programming : Several small procedures that operate on data/states
 - Different from declarative programming like functional, relational or logic programming
 - Examples: C, Basic, Pascal, Fortran (C++, C#, Ada, Prolog, Lisp...)

Procedural programming vs object-oriented programming

- Emphasis on procedures unlike OOP which focuses on behavior
- Data structures [struct, union] – much like classes without methods
- Programming is writing procedures to do something with these structures
- Recursion – calling procedure within itself

Hello World

The last task of last week was to create a program that printed Hello World, most of you managed this – but let us look at this in detail :-)

Hello world!

```
#include <stdio.h>
```

```
int main(int argc, char *argv[]){  
    //My first C program  
    printf("Hello world!\n");  
    return 0; //All OK  
}
```

```
-$ gcc -O2 -Wall -Werror -std=c11 hello.c -o hello
```

```
-$ ./hello
```

VOILA!

We did not talk about this last week, but for those new to Linux:
When you start an application, Linux “doesn’t know” which folder you are in, so you must start with `./` before the name of the executable – which means “search in this folder” :-)

#include

- Order preprocessor to include other file
- It simply reads the named file, then continue in your file
- `< >` implies the system include folders
 - `/usr/include`
 - `/usr/local/include`
 - Folders given by `-i` option on `gcc` command line.
- You can also include your own files by using `" "`
 - `#include "myHeader.h"`
 - `#include "/home/vamsi/myHeader.h"`
 - `#include "../inc/myHeader.h"`

*Important; Good coding practice:
Only include header files (or other
such as "settings"), do NOT
include other source files (.c)!*

*Including .c files will deduct
points on an exam, it is bad
coding practice...*

```
int main(int argc, char *argv[])
```

- The main entry point of your program
- C89 requires it to return an **int** (newer standards can have **void**)
- But you can return void depending on the compiler
- **Return 0** for success and an error-code otherwise
- It can help in scripts with multiple executables

Also supported:
int main (void)
void main (void)

Arguments to main

- **int main()**, **int main(int argc, char* argv[])** – does not enforce any number of arguments or their type
- **int main(void)** – implies that it must be passed with no arguments
- If used
 - argc – Number of arguments
 - argv – array that holds arguments in string format
- Example : \$./hello a1 b2
 - argc == 3
 - argv[0] == './hello'
 - argv[1] == 'a1'
 - argv[2] == 'b2'

printf

- Prints out text to stdout (standard output is typically the screen)
- *print format string*
- Some example formats:
 - %d – int
 - %f – float
 - %s – string
 - %x – hexadecimal number
 - Check man page
- ‘\n’ is used for line break

```
int a=3;  
printf("a is: %d\n", a);
```

gcc inputs

- ‘-o’ to inform where to put the output file
- ‘-std=c11’ to ask the standard that should be used
 - careful : C11 is 2011 and the new standard
 - OpenCL still uses C99
- ‘-Wall’ : Show all warnings
- ‘-Wextra’ : Show even more warnings
- ‘-O2’ : Optimize.
‘-O0’ : Must be turned off for debug.
- ‘-Werror’ : Make all warnings as errors
 - sometimes it is a bad idea

Basics of C programming

Commenting is just like Java

- Block commenting - `/* */`

Example :

```
/* Hello this is  
   and example of  
   multiline block  
   commenting */
```

- Line commenting - `//`

Example :

```
// This is how you can comment  
// multiple lines
```

Line commenting is strictly speaking not part of ANSI C, it was introduced in C99 – but you are allowed to use it for this class

Style guidelines

Using tabs or spaces, and “3 or 4 spaces”-in-a-tab could lead to wars, do what the seniors in the company does...

Important; Good coding practice:
Required for top score on exam

- Good for readability!
- Tab is usually 3 spaces
- Limit the length of the line to 80 characters
- Line breaks are better with an operator at the end

Example :

```
int iNumberOfDaysUntilTheEndOfTheUniverse = 7;
```

```
iNumberOfDaysUntilTheEndOfTheUniverse = iNumberOfDaysUntilTheEndOfTheUniverse -  
iNumberOfDaysLeft + 42;
```

- Indentation is important!
- For more guidelines read **Google code guide**
<https://google.github.io/styleguide/cppguide.html>

Variable scope

- You can have global variable and local variables
- **Global** : declare it once and it is available for all the code that comes next (*not always a good idea...*)
- **Local** : Limited scope usually in the functions

*Important; Good coding practice:
Do not use global variables,
except in some very specific cases!*

*Using global variables instead of
passing arguments to functions
will deduct points on an exam, it
is bad coding practice...*

Variable types

- Declaration is similar to java, but some types have architecture-dependent definitions.
 - char
 - short
 - int
 - long
 - long long
 - float
 - double
- You can have signed and unsigned

inttypes.h

- Makes your code more portable
- unsigned integer: `uint8_t`, `uint16_t`, `uint32_t`
- signed: `int8_t`, `int16_t`
- Read more from the standard documentation

We do not use these during the lectures in this course, but I have used types like this in production code and it has its benefits :-)

C has no native boolean type

- `#include <stdbool.h>`
 - `'bool', 'true', 'false'`
 - Unsafe type
 - `'0'` is false, everything else is true
- Common: using `int` with `'1'` as true and `'0'` as false

Similar to Java

- while
- for
- if/else/else if
- switch
- do while
- No iterators in C! (*for each...*)
- +, -, *, /, +=, %, &&, &, =, ==,
- !=, <, >, <=, ^, !, ~ etc

```
unsigned Int counter = 0;

Counter = 0;
while (counter > 0) {
    Ddd();
    Counter--;
}

For (counter = 5; counter > 0;
counter--) {
    ddd();
}

Counter = NumberOfWowPlayers();
Do {
    ddd();
    counter--; // = 0xffffffff
    counter = counter - 1;
    counter -= 1;
} While (counter > 0);
```


Functions and variables

Functions are used to make the program modular

- A Function is declared with a name, arguments and return type
 - example: `int sum(int a, int b){ return a+b; }`
- The function must be declared before it is used, but not necessarily implemented.

- example :

```
int sum(int a, int b);
```

```
int main(int argc, char *argv[]){  
    int result;  
    result = sum(9,10);  
    return 0;  
}
```

```
int sum(int a, int b){  
    return a+b;  
}
```

- Recursion is supported in C
- Main is also a function!

getchar()

- Simplest input
- Reads a char from keyboard
- returns a character

```
char c = getchar();  
printf("%c is the character", c);
```

- Blocking input – waits forever until a char is seen

Static Array

```
int sorted[10]; // integer array  
int length = sizeof(sorted);
```

- `sizeof()` returns bytes
- Stored on the stack if it is not declared with `static/` as a global array

scanf(format string)

```
char buf[80];
```

```
int num;
```

```
scanf("%s", buf);
```

```
scanf("%d", &num); // Reference to num
```

```
scanf("%d %s", &num, buf);
```

- scanf() returns the number of elements read

Note that reading into a text buffer using scanf("%s") is dangerous – it does not check for number of characters and can result in a buffer overflow (a serious security vulnerability)!

We will learn better functions.

Important; Good coding practice: Buffer overflow on the exam will deduct points and lower your grade. Make a habit on secure coding – many vulnerabilities stem from C programming!

Type casting

- Casting can be “implicit” or “explicit”

```
char c = 'N';
```

```
int d = c; //implicit cast
```

```
float f = (float) d //explicit cast
```

```
printf("%f", f); // output : 78.00000
```

- Casting can be unsafe since C lets most casting through
- Pointers can be casted too – more on this later.

Datatype

- The **datatype** of an object in memory determines the set of values it can have and what operations that can be performed on it
- C is a *weakly* typed language. It allows implicit conversions as well as explicit casting.
- Example : int, char, long etc.
- Sizes are different depending on platform

Operators

- **Operators** specify how an object can be manipulated (eg., numeric vs. string operations).
- Operators can be
 - Unary : ++, --, ~
 - Binary : +, -, *, /, %
 - Ternary : ?:
 - Example -> result = (condition)? (result1):(result2)

```
int bigger(int a, int b){  
    return (a>b) ? a:b;  
}
```

Unary means it uses *ONE* variable. Example: i++;

Binary means it uses *TWO* variables. Example: a + b;

Expressions

- An **expression** in a programming language is a combination of values, variables, operators and functions
- Example :

$y = x + 2;$

$y = x * 3;$

$y = x \% 4;$

Not everyone remembers this last one (we covered it in the TK-classes, and its basic math), the % operator is a “reminder division” (also called modulus) which means it divides X by 4, and the reminder is assigned to y.
9 % 4, y becomes 1 because 9 divided by 4 is 2, and the reminder is then 1...

Variables

- A **variable** is a named link/reference to a value stored in the system's memory or an expression that can be evaluated.

- **Example :**

```
int *a;
```

```
int x,y;
```

```
char z;
```

a, x, y, z are all **variables**

Variable names

- Variable names can contain letters, digits and _
- Variable names can *not* start with digits
- Keywords cannot be used as variable names
- Variable names are case sensitive.
- Examples:
 - Allowed: `int x, X, radiusX, _radius, _circle2`
 - Not Allowed: `int x$, xyz$, 8low, 8high`
 - Better: `int iSize, iCounter;`

The last 2 examples use what we can “Hungarian Notation”, which means that we use the **TYPE** as part of the name of a variable. This helps the reader understand that `iCounter` is in fact an `int`.

Hungarian Notation is recommended for readability :-)

Data type categories

- Numeric (int, float, double)
- Character (char)
- User defined (struct, union)

Numeric data types

- short
 - int
 - long
 - float
 - double
-
- Signed Vs Unsigned
 - Range depends on signed and unsigned

Sizes of variables

- Sizes are dependent on compiler and machine
- Guaranteed :
 - `sizeof(char) < 2 <= sizeof(short) <= sizeof(int) <= 4 <= sizeof(long)`
 - `sizeof(char) <= sizeof(short) <= sizeof(float) <= sizeof(double)`
- These are important for overflow.
- What are the sizes on your machine?[exercise]

(Integer) Overflow

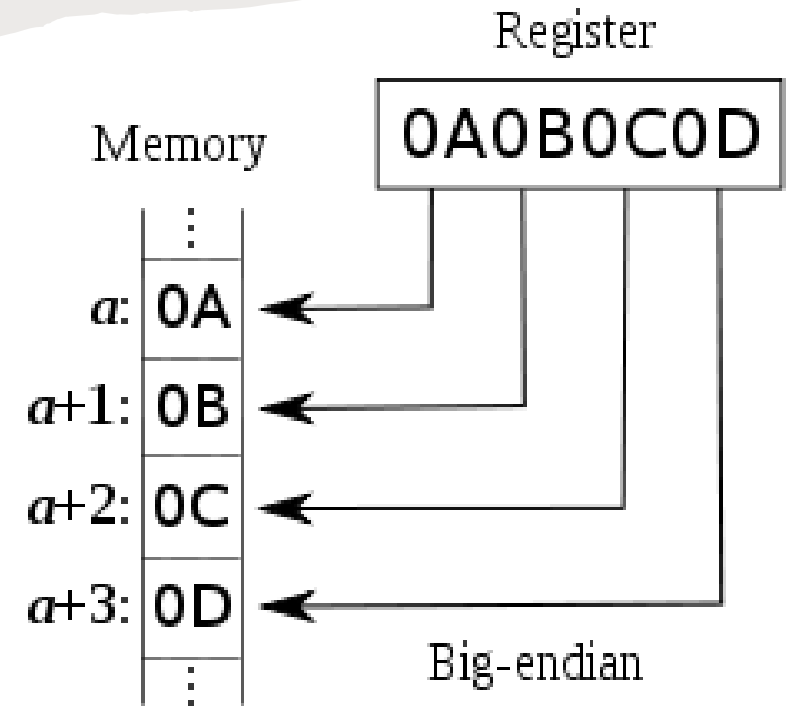
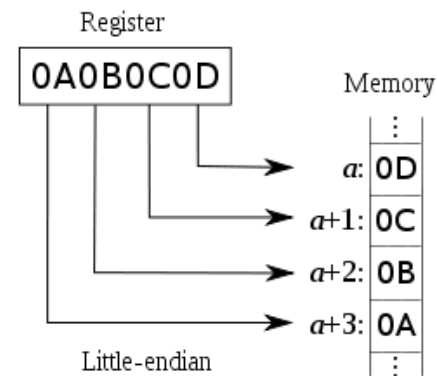
- Limited number of bits to store the values
- Unsigned has bigger range:
 - 8 bit : 0 to 255
 - 16 bit : 0 to 65535
 - so on
- Signed has lower range because of usage of one bit for sign:
 - 8 bit : -128 to 127
 - 16 bit : -32,768 to 32,767
 - so on
- When you add two integers – C doesn't check for overflow
 - Example : `short a = 32767, b = 32767;`
`short c = a +b ;//Oh well! this will go to hell!`
- Overflow can be expensive – Ariane 5 rocket!
- <https://www.bbc.com/future/article/20150505-the-numbers-that-lead-to-disaster>

Literals

- Literals are literal/fixed values assigned to variable or used directly in expressions
- Examples:
 - Integer : `4`, `5UL`, `0xC`, `071`
 - Floating point : `3.1415`, `3.14F`, `2.0`, `3.1415265359L`
 - Character : `'A'`, `'\x41'`, `'\0101'`
 - String : `"this is a string"`
 - Enumeration : `enum eAnswers {NO, YES, MAYBE};`

Endian-ness (mess...)

- Little Endian – when lower memories hold low significant bits (Intel x86 etc.)
- Big Endian – When lower memories hold most significant bits (PowerPC, 68K, 88K and networks)



This is only important when you either store data in a file that will be transferred to another system, or when you send data over a network...

Operators and Control flow

Operators

- Arithmetic operators
- Relational operators
- Logical operators
- Bitwise operators
- Assignment operators

Arithmetic operators

- Addition - '+'
- Subtraction – '-'
- Multiplication – '*'
- Division – '/' => quotient in integer case
- Modulus – '%' => of course usually used on int
- Increment – '++'
- Decrement – '--'

Relational operators

- Greater than : >
- Greater than or equal to: >=
- Lesser than : <
- Lesser than or equal to : <=
- Equal to : ==
- Not equal to : !=
- Avoid using ==/ != for float/double, instead use an expression
 - $\text{abs}(f1-f2) < e$
- All return 0/1 !

Logical operators

- And - &&
- Or - ||
- Not - !
- Xor - ^
- Usage of x^1 for alternative on/off flag
- Caution :

The expressions are evaluated from left to right, short circuited.

Example : `(4!=5) || ((c=getchar())=='n')`

This will never wait for you to type n

Bitwise operators

- AND - &
- OR - |
- XOR - ^
- Left shift - <<
- Right shift - >>

Blocks

```
{  
    int a= 20;  
    //This is block1  
    {  
        //This is an inner block  
        printf("%d", a);  
    }  
}  
  
{  
    //This is a different block  
    printf("%d", a); //error a undefined  
}
```


Blocks

- Compiled as a single unit
- Allows variable declaration
- A variable in a block is accessible over the entire block including all the inner blocks
- If a variable using the same name is defined in the inner block, then the first variable's scope is not valid in the inner block. It will be replaced with the new variable.

```
{  
    int a = 20;  
    //a here is 20  
    {  
        //a here changes to 40  
        int a = 40;  
        printf("%d", a);  
    }  
    printf("%d", a); //a here is 20  
}
```

Conditional expression

- An expression or a collection of expressions that evaluates to false/true
- Expression = zero implies false, non zero implies true
- Example :

`(A > 5)`

`(a<3) && (b>5)`

Control structures

- if
- for
- while
- do - while
- switch
- break
- continue

if

- if , else if, else
- Same as java
- Allows for empty statements, blocks and single lines.

```
if (i < 10) {  
    ...  
}
```

Switch

```
switch (iVar) {  
    case 0:  
        statements;  
        break;  
    case 1 :  
        statements;  
        break;  
    default :  
        statements;  
}
```

- Works for int and char!

for

- Similar as Java

for (initializations; conditions; processing) ...

```
for (int i=0; i<10; i++) {  
    ...  
}
```

```
for (;;) {  
    ...  
    if (...) break;  
}
```

```
for (;;) ;
```

Last one there is a joke... ;-)

while

```
while (condition) {  
    //do stuff here  
}
```

- Doesn't guarantee an execution of stuff!
- The condition is evaluated first and then stuff are executed

do-while

- Enables for at least one guaranteed execution

```
do{  
    //do stuff here  
}while(condition)
```


continue and break

- Control keywords used to manipulate loop
- Continue starts the loop again from the top
- Break ends the loop and moves execution to after the loop

```
do{  
    //do stuff here  
    if (specialcase) continue;  
    else (secondcondition) break;  
}while(condition)
```

Sequencing can lead to confusing behavior...

```
a = a++; // Nothing happens;...  
a = ++a;  
a++; ++a;
```

```
#include <stdio.h>  
  
int a() {puts("2"); return 2;}  
int b() {puts("3"); return 3;}  
  
void main(void) {  
    int x=a() + b();  
    printf("%d\n",x);  
}
```

Lab exercises

- **Goal for today is to experiment with variables and control flow**
- **Some students feel the exercises for the next few weeks in this course are little concrete – that is because we can not “do much” with just variables and control. Therefore each lab task is short, and you will have to experiment within the parameters of those simple tasks :-)**
- **I recommend using the time only experimenting on the simple tasks, some students wish to make a large program already, but restrain yourselves and learn how to walk before you try to fly...**

That was all the basics covered...

So next week we can start with the FUN; pointers :-)



Høyskolen
Kristiania